



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

Dynamic Programming

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Content

- Concept of Dynamic Programming
 - Top-down approach
 - Bottom-up approach
- Fibonacci number
- Binomial Coefficient
- Sum of Subset
- 0/1 Knapsack Problem
- Matrix Chain Multiplication
- Longest Common Subsequence (LCS)

Concept of dynamic programming (DP)

- การโปรแกรมเชิงพลวัต (Dynamic Programming) เป็นเทคนิคการการแก้ปัญหาโดยมีแนวคิดในการแบ่งปัญหาออกเป็นปัญหาย่อย (sub problems) คล้ายกับการแบ่งแยกและเอาชนะแต่ใช้วิธีการจำคำตอบ (memorization) เพื่อลดเวลาคำนวณที่ซ้ำซ้อน
- ประสิทธิภาพของเทคนิคการโปรแกรมเชิงพลวัตจึงมาจากวิธีการจำคำตอบของปัญหาย่อยที่ได้เตรียมไว้ล่วงหน้าและความสัมพันธ์ระหว่างคำตอบปัจจุบัน (current solution) และคำตอบย่อยของมัน

Solution = sub structure + memorization

ตัวอย่าง Computing Fibonacci numbers

- ลำดับเลขอนุกรมไฟโบนัคชี (Fibonacci numbers)

1, 1, 2, 3, 5, 8, 13, 21,

เราสามารถเขียนฟังก์ชันเพื่อคำนวณเลขไฟโบนัคชีลำดับที่ n ได้

$$fib(n) = \begin{cases} 1, & n = 0, 1 \\ fib(n - 1) + fib(n - 2), & n > 1 \end{cases}$$

```
{ if (n < 2) return 1;
  return f(n-2) + f(n-1);
}
```

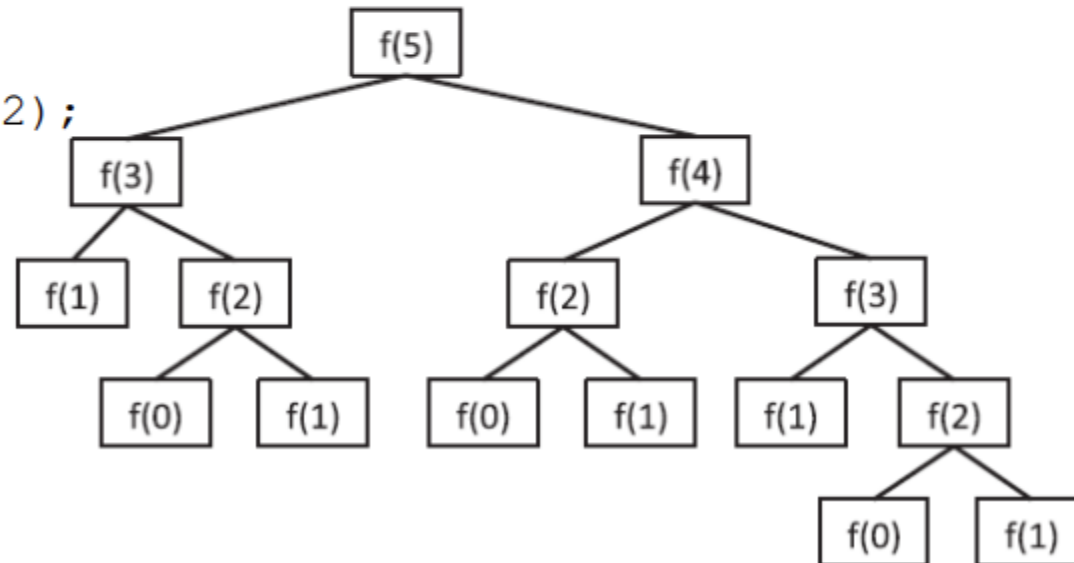


แนวคิดของ Dynamic Programming (Top-Down)

- ใช้หน่วยความจำส่วนหนึ่งจดจำคำตอบที่เคยคำนวณมาก่อนหน้าเพื่อลดเวลาการทำงาน

```
int M[100];
f(n)
{ if(n <=1)
  return n;
  else if(M[n] != 0)
    return M[n];
  else {
    M[n] = f(n-1)+f(n-2);
    return M[n];
  }
}
```

M					
0	1	2	3	4	5
0	1				



แนวคิดของ Dynamic Programming (Bottom Up)

- ใช้หน่วยความจำส่วนหนึ่งจดจำคำตอบที่เคยคำนวณมาก่อนหน้าเพื่อลดเวลาการทำงาน

$f(n)$

```

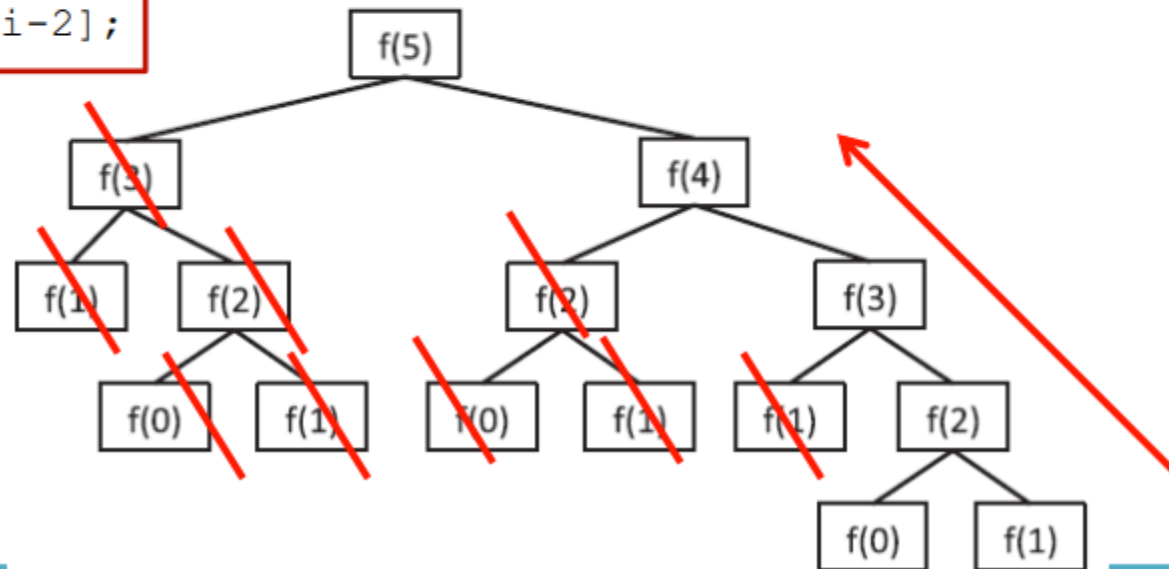
{  int i;
    int F[100] = {0};
    F[0] = 0;
    F[1] = 1;

    for(i = 2; i<=n; i++)
        F[i] = F[i-1] + F[i-2];

    return F[n];
}
    
```

M

0	1	2	3	4	5
0	1				



ปัญหาที่เหมาะสมกับ Dynamic Programming

- ต้องสามารถนิยามคำตอบของปัญหาในรูปของโครงสร้างย่อยซึ่งประกอบไปด้วยคำตอบ ปัจจุบัน (current solution) และคำตอบก่อนหน้า (previous solution) ตัวอย่างเช่น คำตอบของ $f(n)$ สามารถนิยามเป็น $f(n-1) + f(n-2)$
- มีปัญหาย่อยทับซ้อนกัน (overlap sub-problem) เช่น เมื่อเราคำนวณ $F(4) = F(3) + F(2)$ และ $F(3) = F(2) + F(1)$ จะเห็นว่า $F(2)$ ถูก คำนวณซ้ำซ้อน

การเตรียมคำตอบของปัญหาย่อย (Memorization)

- เทคนิค dynamic programming มักจะใช้อาร์เรย์ (array) หรือตาราง (table) ที่ถูก ออกแบบ ซึ่งสัมพันธ์กับโครงสร้างย่อยของปัญหาเพื่อจำคำตอบก่อนหน้านี้
- ดังนั้นเวลาที่ใช้ในการหาคำตอบของปัญหาก็คือการปรับปรุงค่า (update) ในตารางเมื่อ ได้ คำตอบในแต่ละช่องภายในอาร์เรย์ดังกล่าว
- กระบวนการหาคำตอบมี 2 แบบ ดังนี้
 - แบบ top-down
 - แบบ bottom-up (Efficiency improvement)

Binomial Coefficient

- binomial coefficient $C(n, k)$ หรือ $\binom{n}{k}$ คือจำนวนของเซตย่อยของข้อมูลขนาด k จากเซตข้อมูลทั้งหมด n จำนวน ($0 \leq k \leq n$).

$$C(n, k) = \frac{n!}{(n-k)! * k!}$$

เช่น $C(3,2) = 3! / (3-2)! * 2! = 6 / 1 * 2 = 3$

- การหาค่า $C(n, k)$ สามารถเขียนในรูปของ recursion ได้

$$C(n, k) = C(n - 1, k - 1) + C(n - 1, k) \quad \text{for } n > k > 0$$

และ

$$C(n, 0) = C(n, n) = 1$$

Binomial Coefficient

$$C(n, k) = C(n-1, k-1) + C(n-1, k) \quad \text{for } n > k > 0$$

และ $C(n, 0) = C(n, n) = 1$

$$\begin{aligned} C(n, k) &= n! / (k! * (n-k)!) \\ &= (n-1)! * n / (k! * (n-k)!) \\ &= (n-1)! * n / ((k-1)! * k * (n-k-2)! * (n-k-1) * (n-k)) \\ &= (n-1)! / ((k-1)! * (n-k-1)!) * (n / k * (n-k)) \\ &= [(n-1)! / ((k-1)! * (n-k-1)!)] * (1/(n-k) + 1/k) \\ &= (n-1)! / ((k-1)! * (n-k)!) + (n-1)! / (k! * (n-k-1)!) \\ &= C(n-1, k-1) + C(n-1, k) \end{aligned}$$

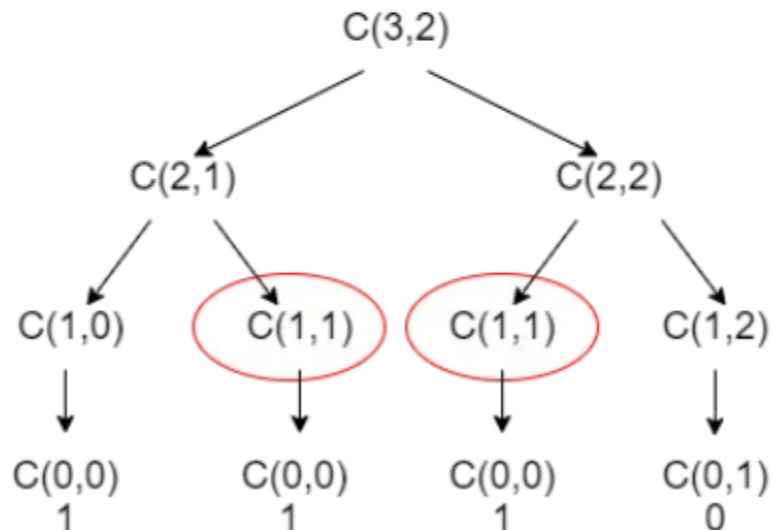
ตัวอย่าง $C(3, 2) = C(2, 1) + C(2, 2) = 2 + 1 = 3$

$$\begin{aligned} C(2, 1) &= C(1, 0) + C(1, 1) \\ &= 1 + 1 = 2 \end{aligned}$$

$$C(2, 2) = 1$$

Recursive approach

```
int recursiveBinomialCoefficientMethod(int n,int k)
{
    if(k == n or k == 0)
        return 1;
    return (recursiveBinomialCoefficientMethod(n-1,k-1)+recursiveBinomialCoefficientMethod(n-1,k));
}
```



Time Complexity: $O(2^n)$

Dynamic Programming approach

- เตรียม table เพื่อเก็บคำตอบของ $C(n, k)$ ในทุกๆ ค่าของ n และ k ที่เป็นไปได้
- เลือกวิธีเติมคำตอบลงใน table (bottom-up หรือ top-down)

	0	1	2	...	k-1	k
0	1					
1	1	1				
2	1	2	1			
⋮						
k	1					1
⋮						
n-1	1				$C(n-1, k-1)$	$C(n-1, k)$
n	1				$\swarrow$	$\downarrow$ $C(n, k)$

การคำนวณ compute $C(6, 4)$

n/k	0	1	2	3	4	5	6	7
0	1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0
2	0	1	1	0	0	0	0	0
3	0	1	2	1	0	0	0	0
4	0	1	3	3	1	0	0	0
5	0	1	4	6	4	1	0	0
6	0	1	5	10	10	5	1	0
7	0	1	6	15	20	15	6	1

$$C(n, k) = C(n - 1, k - 1) + C(n - 1, k) \\ \text{for } n > k > 0$$

$$C(n, 0) = C(n, n) = 1$$

$$C(6, 4) = C(5, 3) + C(5, 4)$$

$$C(5, 3) = \text{table}[5, 3] \text{ หรือหาคำตอบจาก} \\ C(5, 3) = C(4, 2) + C(4, 3)$$

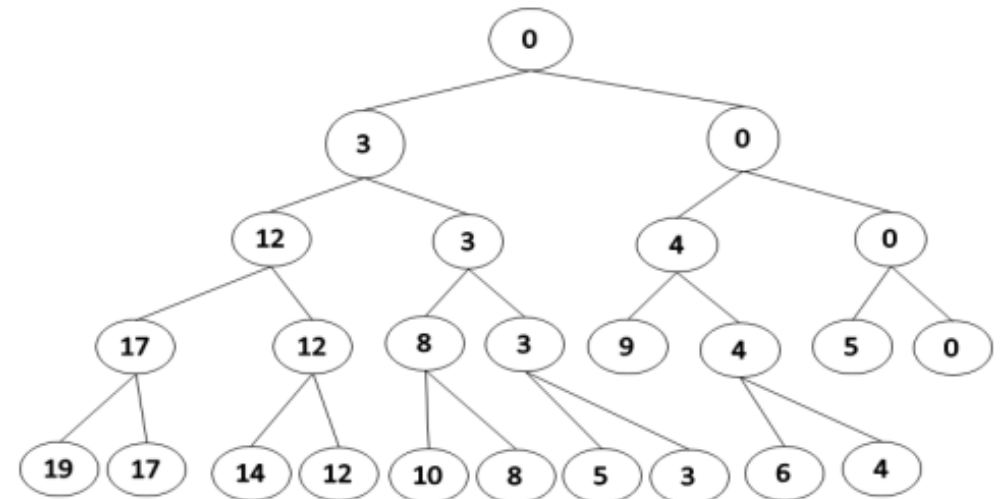
$$C(5, 4) = \text{table}[5, 4] \text{ หรือหาคำตอบจาก} \\ C(5, 4) = C(4, 3) + C(4, 4)$$

Workshop

- จงเขียนฟังก์ชันเพื่อคำนวณ binomial coefficient ของ n, k โดยใช้ วิธี bottom-up และ top down dynamic programming

Subset of sum

- ให้ $A = \{3, 4, 5, 2\}$ ต้องการหาว่ามีเซตย่อยที่มีผลรวมเท่ากับ $k=6$ หรือไม่
- จากตัวอย่างพบว่า $\{4, 2\}$ จะมีผลรวมเท่ากับ 6
- โดยทั่วไปสามารถหาคำตอบด้วยวิธี brute force หรือ backtracking อย่างไรก็ตามพบว่ามีวิธีดังกล่าว อาจใช้เวลาค่อนข้างมากเนื่องจากต้องการ generate สถานะของคำตอบที่เป็นไปได้ทั้งหมด
- โดยหลักการของ Dynamic programming
เราสามารถนิยามโครงสร้างย่อย (Sub structure)
เพื่อหาคำตอบ $\text{Sum}(n, k)$



Subset of sum

- $\text{Sum}(n, k)$ อาจนิยามคำตอบโดยพิจารณาจากสมาชิกตัวสุดท้ายของ A ($A[n-1]$)
 - หากคำตอบมี $A[n-1]$ เป็นส่วนหนึ่งของคำตอบ ดังนั้น
 - $\text{Sum}(n, k) = \text{Sum}(n-1, k - A[n-1])$
 - หากคำตอบไม่มี $A[n-1]$ เป็นส่วนหนึ่งของคำตอบ ดังนั้น
 - $\text{sum}(n, k) = \text{sum}(n-1, k)$

$$\text{Sum}(n, k) = \text{Sum}(n-1, k) \text{ or } \text{Sum}(n-1, k - A[n-1])$$

- $A = \{3, 4, 5, 2\}$ และ $k = 6$
 $\text{Sum}(4, 6) = \text{Sum}(3, 6) \text{ or } \text{Sum}(3, 6 - 2)$

Subset of sum



$A[n-1]/k$	0	1	2	3	4	5	6
$\{\}$	T	F	F	F	F	F	F
$\{3\}$	T						
$\{3, 4\}$	T						
$\{3, 4, 5\}$	T				Sum(3, 4)		Sum(3, 6)
$\{3, 4, 5, 2\}$	T						Sum(4, 6)

$$\text{Sum}(n, 0) = T$$

$$\text{Sum}(0, k) = F$$

$$\text{Sum}(4, 6) = \text{Sum}(3, 6) \text{ or } \text{Sum}(3, 6-2)$$

Workshop

- จงเขียนฟังก์ชันเพื่อคำนวณ Subset of sum ของ $\text{Sum}(\text{int } n, \text{int } k)$ โดยใช้ วิธี bottom-up และ top down dynamic programming

0/1 knapsack problem

- ถุงใส่ของรับน้ำหนัก W_t กิโลกรัม มีของ n ชิ้น แต่ละชิ้นมีน้ำหนัก (w_i) และมูลค่า (v_i) ที่แตกต่างกัน หาวิธีเลือกของใส่ถุงเพื่อให้มีมูลค่ารวมมากที่สุด และน้ำหนักรวมไม่เกิน W_t

w_i	v_i
1	1
3	4
4	5
5	7

$w_t = 7$

$$\max \sum_{i \in T} v_i \quad \text{subject to} \quad \sum_{i \in T} w_i \leq W_t$$

0/1 knapsack problem

w_i	v_i
1	1
3	4
4	5
5	7
$wt = 7$	

weight = 1

value = 1

weight = $1+3 = 4$ value = $1+4 = 5$

weight = 3 value = 4

weight = $3+4 = 7$ value = $4+5 = 9$

Optimal
solution



Optimal sub-structure of 0/1 knapsack

จากนิยามของปัญหา พบว่าคำตอบของปัญหา สามารถพิจารณาจาก 2 กรณี คือ

1. ถ้าน้ำหนักของไอเท็มไม่เกินน้ำหนักของถุงที่ยังรับได้ เราก็จะใส่ไอเท็มเข้าไปในถุง ดังนั้นมูลค่ารวมก็จะเพิ่มขึ้น
2. ถ้าน้ำหนักเกินก็ไม่ใส่ไอเท็มเข้าไปในถุง มูลค่ารวมก็จะเท่ากับไอเท็มที่ถูกใส่ก่อนหน้านี้

ดังนั้นหากรับประกันได้ว่ามูลค่ารวมของไอเท็มที่ใส่ก่อนหน้านี้มากที่สุด การใส่ไอเท็มนี้เข้าไปก็จะทำให้ได้ผลรวมของมูลค่ามากที่สุดด้วย

กำหนดให้ M_{ij} หมายถึงมูลค่ารวมที่มากที่สุดเมื่อใส่ของชิ้นที่ i ลงไปในถุงที่รับน้ำหนักได้ j ดังนั้น

$$M_{ij} = \begin{cases} \max\{V_i + M_{i-1,j-w_i}, M_{i-1,j}\}, & w_i \leq j \\ M_{i-1,j}, & \text{otherwise} \end{cases}$$

$W_t = 7$

$V: 1\ 4\ 5\ 7$

$W: 1\ 3\ 4\ 5$

$$M_{ij} = \begin{cases} \max\{V_i + M_{i-1, j-w_i}, M_{i-1, j}\}, & w_i < j \\ M_{i-1, j}, & \text{otherwise} \end{cases}$$

เริ่มต้นจากการเติมค่า 0 ให้กับคอลัมน์ 0 และแถว 0

$i \setminus j$	0	1	2	3	4	5	6	7
(v, w)	0	0	0	0	0	0	0	0
(1, 1)	0							
(4, 3)	0							
(5, 4)	0							
(7, 5)	0							

Wt = 7

V : 1 4 5 7

W: 1 3 4 5

$$M_{ij} = \begin{cases} \max\{V_i + M_{i-1,j-w_i}, M_{i-1,j}\}, & w_i < j \\ M_{i-1,j}, & \text{otherwise} \end{cases}$$

$$M_{11} = \max\{1 + M_{0,0}, M_{0,1}\} = 1$$

$$M_{12} = \max\{1 + M_{0,1}, M_{0,2}\} = 1$$

$$M_{13} = \max\{1 + M_{0,2}, M_{0,3}\} = 1$$

i \ j	0	1	2	3	4	5	6	7
(v, w)	0	0	0	0	0	0	0	0
(1, 1)	0	1	1	1	1	1	1	1
(4, 3)	0							
(5, 4)	0							
(5, 7)	0							

Wt = 7

V : 1 4 5 7

W: 1 3 4 5

$$M_{ij} = \begin{cases} \max\{V_i + M_{i-1, j-w_i}, M_{i-1, j}\}, & w_i < j \\ M_{i-1, j}, & \text{otherwise} \end{cases}$$

$$M_{21} = M_{11} = 1$$

$$M_{22} = M_{12} = 1$$

$$M_{23} = \{4 + M_{10}, M_{13}\} = 4$$

$$M_{24} = \{4 + M_{11}, M_{14}\} = 5$$

i \ j	0	1	2	3	4	5	6	7
(v, w)	0	0	0	0	0	0	0	0
(1, 1)	0	1	1	1	1	1	1	1
(4, 3)	0	1	1	4	5	5	5	5
(5, 4)	0							
(7, 5)	0							

Wt = 7

$$M_{ij} = \begin{cases} \max\{V_i + M_{i-1, j-w_i}, M_{i-1, j}\}, & w_i < j \\ M_{i-1, j}, & \text{otherwise} \end{cases}$$

V : 1 4 5 7

W: 1 3 4 5

$$M_{47} = \max \{ 7 + M_{32}, M_{37} \} = \max \{ 7 + 1, 9 \} = 9$$

i \ j	0	1	2	3	4	5	6	7
(v, w)	0	0	0	0	0	0	0	0
(1, 1)	0	1	1	1	1	1	1	1
(4, 3)	0	1	1	4	5	5	5	5
(5, 4)	0	1	1	4	5	6	6	9
(7, 5)	0	1	1	4	5	7	8	9

Workshop

- จงเขียนฟังก์ชันเพื่อคำนวณ 0/1 Knapsack ของ knapsack (int Wt, int w[], int v[], int n) โดยใช้ วิธี bottom-up และ top down dynamic programming

Matrix Chain Multiplication

- กำหนดให้เมตริกซ์ A มีขนาด $n \times m$ และเมตริกซ์ B มีขนาด $m \times p$ ดังนั้น $C = AB$ จะได้ผลลัพธ์เป็นเมตริกซ์ C มีขนาด $n \times p$ ซึ่งหากใช้วิธีการคำนวณโดยตรง จำนวนครั้งในการคูณจะเท่ากับ $n \times m \times p$ ครั้ง
- หากขนาดของเมตริกซ์ AB เท่ากัน $n \times n$ จะต้องคูณกัน n^3 ครั้ง
- สมมติมีเมตริกซ์จำนวน n ชุด $A_1, A_2, \dots, A_n$ ขนาดมิติอาจไม่เท่ากัน จงหาลำดับการคูณเมตริกซ์ซึ่งทำให้ผลรวมของจำนวนการคูณน้อยที่สุด ตัวอย่างเช่น 4 เมตริกซ์
 $A_1: [10 \times 100]$, $A_2: [100 \times 5]$, $A_3: [5 \times 50]$, $A_4: [50 \times 1]$ หากนำมาคูณกันแบบเมตริกซ์ $A_1A_2A_3A_4$ จะได้เมตริกซ์ผลลัพธ์เป็น $[10 \times 1]$

ลำดับผลคูณเมตริกซ์

$$(A_1(A_2(A_3A_4)))$$

- $A_{34} = A_3A_4$, 250 mults, result is 5 by 1
- $A_{24} = A_2A_{34}$, 500 mults, result is 100 by 1
- $A_{14} = A_1A_{24}$, 1000 mults, result is 10 by 1
- **Total is 1750**

$$((A_1A_2)(A_3A_4))$$

- $A_{12} = A_1A_2$, 5000 mults, result is 10 by 5
- $A_{34} = A_3A_4$, 250 mults, result is 5 by 1
- $A_{14} = A_{12}A_{34}$, 50 mults, result is 10 by 1
- **Total is 5300**

$$(((A_1A_2)A_3)A_4)$$

- $A_{12} = A_1A_2$, 5000 mults, result is 10 by 5
- $A_{13} = A_{12}A_3$, 2500 mults, result is 10 by 50
- $A_{14} = A_{13}A_4$, 500 mults, results is 10 by 1
- **Total is 8000**

$$((A_1(A_2A_3))A_4)$$

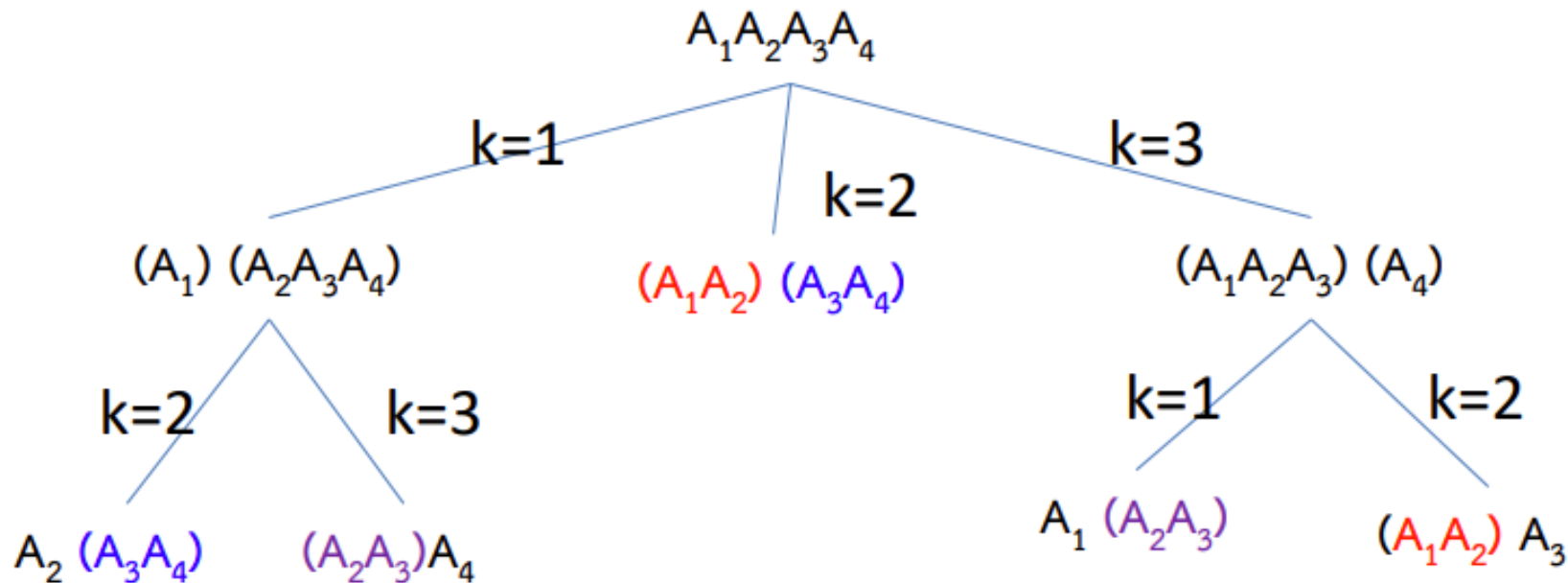
- $A_{23} = A_2A_3$, 25000 mults, result is 100 by 50
- $A_{13} = A_1A_{23}$, 50000 mults, result is 10 by 50
- $A_{14} = A_{13}A_4$, 500 mults, results is 10 by
- **Total is 75500**

$$(A_1((A_2A_3)A_4))$$

- $A_{23} = A_2A_3$, 25000 mults, result is 100 by 50
- $A_{24} = A_{23}A_4$, 5000 mults, result is 100 by 1
- $A_{14} = A_1A_{24}$, 1000 mults, result is 10 by 1

Optimal substructure

คำตอบของผลคูณเมตริกซ์ $A_1 \dots A_n$ สามารถแบ่งเป็นโครงสร้างย่อย โดยการแบ่งลำดับการคูณของเมตริกซ์ออกเป็น 2 ส่วน คือ $(A_1 \dots A_k)$ และ $(A_{k+1} \dots A_n)$ จากนั้นรวมคำตอบที่ได้จากการคูณชุดเมตริกซ์ $A_1 \dots A_k$ และ $A_{k+1} \dots A_n$ มารวมกันอีกครั้ง ซึ่งถ้าหากในแต่ละครั้งของการแบ่งชุดอาร์เรย์รับประกันว่าได้จำนวนการคูณน้อยที่สุด ก็จะทำให้คำตอบที่ต้องการน้อยที่สุดด้วย เรียกว่า **optimal substructure**



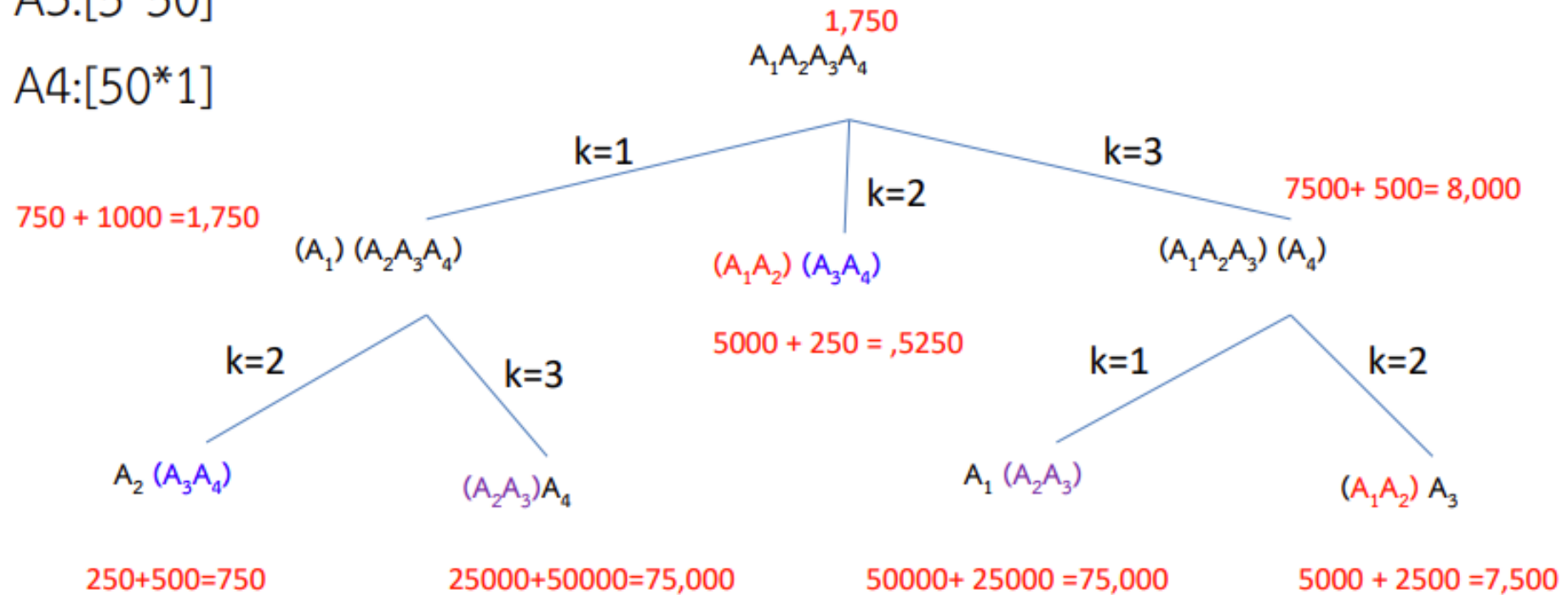
A1: [10*100]

A2: [100*5]

A3:[5*50]

A4:[50*1]

จะเห็นได้ว่าหากเราเลือกคำตอบย่อยของชุดเมตริกซ์ที่ให้จำนวนการคูณที่น้อยที่สุดทุกครั้ง ก็จะทำให้ผลรวมของคำตอบย่อยได้น้อยที่สุดไปด้วย



กำหนดให้ $N_{i,j}$ เป็นจำนวนคูณที่น้อยที่สุดจากเมตริกซ์ $A_i, \dots, A_j$ สามารถเขียนเป็นสมการต่อไปนี้

$$N_{i,j} = \min_{i \leq k < j} \{N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\},$$



$$N_{14} = \min \{ \begin{aligned} &n_{11} + n_{24} + 10*100*1, \\ &n_{12} + n_{34} + 10*5*1, \\ &n_{13} + n_{44} + 10*50*1 \end{aligned} \}$$

$$N_{13} = \min \{ \begin{aligned} &n_{11} + n_{23} + 10*100*50, \\ &n_{12} + n_{33} + 10*5*50 \end{aligned} \}$$

$$N_{24} = \min \{ \begin{aligned} &n_{22} + n_{34} + 100*5*1, \\ &n_{23} + n_{44} + 100*50*1 \end{aligned} \}$$

$$N_{23} = n_{22} + n_{33} + 100*5*50$$

$$N_{34} = n_{33} + n_{44} + 5*50*1$$

$$N_{12} = n_{11} + n_{22} + 10*100*5$$

$$N_{i,j} = \min_{i \leq k < j} \{ N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1} \},$$

เมื่อ $k = 1$

เมื่อ $k = 2$

เมื่อ $k = 3$

เมื่อ $k = 1$

เมื่อ $k = 2$

เมื่อ $k = 2$

เมื่อ $k = 3$

เมื่อ $k = 2$

เมื่อ $k = 3$

เมื่อ $k = 3$

การเตรียมคำตอบของปัญหา matrix chain

	1	2	3	4
1	0	N12	N13	N14
2		0	N23	N24
3			0	N34
4				0

$$N_{i,j} = \min_{i \leq k < j} \{N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\},$$

	1	2	3	4
1	0	5000		
2		0		
3			0	
4				0

$$N_{12} = n_{11} + n_{22} + 10 * 100 * 5 = 5000$$

	1	2	3	4
1	0	5000		
2		0	25000	
3			0	
4				0

$$N_{23} = n_{22} + n_{33} + 100 * 5 * 50 = 25000$$

	1	2	3	4
1	0	5000		
2		0	25000	
3			0	250
4				0

$$N_{34} = n_{33} + n_{44} + 5 \cdot 50 \cdot 1 = 250$$



	1	2	3	4
1	0	5000	7500	
2		0	25000	
3			0	250
4				0

$$N_{13} = \min \left\{ \begin{array}{ll} 0 + 25000 + 10 \cdot 100 \cdot 50 & \text{เมื่อ } k = 1 \\ 5000 + 0 + 10 \cdot 5 \cdot 50 & \text{เมื่อ } k = 2 \end{array} \right\}$$

	1	2	3	4
1	0	5000	7500	
2		0	25000	750
3			0	250
4				0

$$N_{24} = \min \left\{ \begin{array}{ll} 0 + 250 + 100 \cdot 5 \cdot 1 & \text{เมื่อ } k = 2 \\ 25000 + 0 + 100 \cdot 50 \cdot 1 & \text{เมื่อ } k = 3 \end{array} \right\}$$

Dynamic Programming Solution



	1	2	3	4
1	0	5000	7500	1750
2		0	25000	750
3			0	250
4				0

$$\begin{aligned}
 N14 = \min\{ & 0 + 750 + 10 \cdot 100 \cdot 1 \\
 & 5000 + 250 + 10 \cdot 5 \cdot 1 \\
 & 7500 + 0 + 10 \cdot 50 \cdot 1 \} \\
 = & 1,750
 \end{aligned}$$

เมื่อ $k = 1$

เมื่อ $k = 2$

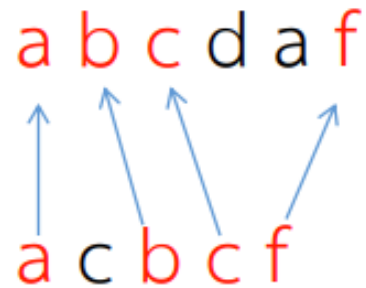
เมื่อ $k = 3$

Workshop

- จงพัฒนาโปรแกรมเพื่อรับขนาดของเมตริกซ์จำนวน n ชุด จากนั้นค้นหาจำนวนครั้งในการ คูณเมตริกซ์เหล่านี้น้อยที่สุด โดยวิธีการ top-down dynamic programming and bottom up dynamic programming

Longest Common Subsequence (LCS)

กำหนดให้ A และ B เป็นสายอักขระ (string) ซึ่งอาจมีความยาวไม่เท่ากัน ต้องการหาลำดับ
ร่วมที่ยาวที่สุดของ A และ B ตัวอย่างเช่น จากตัวอย่างด้านล่าง ลำดับร่วมที่ยาวที่สุดคือ 4

The diagram illustrates the Longest Common Subsequence (LCS) between two strings. The top string is 'a b c d a f' and the bottom string is 'a c b c f'. Blue arrows point from the bottom string to the top string, indicating the matching characters: 'a' (bottom) to 'a' (top), 'c' (bottom) to 'c' (top), 'b' (bottom) to 'b' (top), and 'c' (bottom) to 'c' (top). The common subsequence is 'a c b c', which has a length of 4.

a b c d a f
↑ ↑ ↑ ↑
a c b c f

วิธีการของอัลกอริทึม brute force ก็คือการเช็คทุกลำดับร่วมที่เป็นไปได้ ซึ่งจะเห็นได้ว่าการ
ทำงานที่ซ้ำซ้อนและใช้เวลาในการประมวลผลค่อนข้างนาน

Optimal substructure of LCS

กำหนดให้สายอักขระ X และ Y มีความยาวคือ m และ n ตามลำดับ

ให้ $L(m, n)$ แทนความยาวของลำดับร่วมที่มากที่สุดระหว่าง X และ Y ซึ่งสามารถพิจารณาจาก 2 กรณี ดังนี้

กรณีที่ 1 หาก $X_m = Y_n$ แล้ว

$$L(m, n) = 1 + L(m-1, n-1)$$

a b c d a f
a c b c f

กรณีที่ 2 หาก $X[m] \neq Y[n]$ แล้ว

$$L(m, n) = \max\{L(m, n-1), L(m-1, n)\}$$

a b c d a
a c b c

Basecase

$$L(i, 0) = 0, 0 \leq i \leq m$$

$$L(0, j) = 0, 0 \leq j \leq n$$

$$L(m, n) = \begin{cases} 1 + L(m-1, n-1), & X[m] = Y[n] \\ \max\{L(m, n-1), L(m-1, n)\}, & \text{otherwise} \end{cases}$$

a b c d a f

a c b c f

$$L(m, n) = \begin{cases} 1 + L(m-1, n-1), & X[m] = Y[n] \\ \max \{L(m, n-1), L(m-1, n)\}, & \text{otherwise} \end{cases}$$

$$L(1, 1) = 1 + L(0, 0) = 1$$

$$L(1, 2) = \max \{L(1, 1), L(0, 1)\} = 1$$

1 → n

		a	b	c	d	a	f
	0	0	0	0	0	0	0
1	a	0	1	1			
	c	0					
	b	0					
	c	0					
m	f	0					

↓

Workshop

- จงพัฒนาโปรแกรม longest common subsequence โดยวิธีการ top-down dynamic programming and bottom up dynamic programming